```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error
df = pd.read_csv("uber.csv")
df.head()
df.isnull().sum()
# Clean missing values
df.dropna(inplace=True)
df.drop(['Unnamed: 0', 'key'], axis=1, inplace=True, errors='ignore')
df.isnull().sum()
# ========================================
# EXTRA: VISUALISE OUTLIERS BEFORE CLEANING
# ========================================
plt.figure(figsize=(12, 5))

# Subplot 1: Box plot for fares
plt.subplot(1, 2, 1)
sns.boxplot(x=df['fare_amount'], color='salmon')
plt.title('1. Box Plot: Finding Fare Outliers')
plt.xlabel('Fare Amount ($)')

# Subplot 2: Scatter plot for GPS points
plt.subplot(1, 2, 2)
plt.scatter(df['pickup_longitude'], df['pickup_latitude'], alpha=0.5, color='blue')
plt.title('2. Scatter Plot: GPS Coordinate Outliers')
plt.xlabel('Pickup Longitude')
```

```
plt.ylabel('Pickup Latitude')
plt.tight_layout()
plt.show()
# ========================================
# TASK 1: PRE-PROCESSING & FEATURE ENGINEERING
# ========================================
df['pickup_datetime'] = pd.to_datetime(df['pickup_datetime'])
df['hour'] = df['pickup_datetime'].dt.hour
df['day'] = df['pickup_datetime'].dt.day
df['month'] = df['pickup_datetime'].dt.month
df['year'] = df['pickup_datetime'].dt.year
df['weekday'] = df['pickup_datetime'].dt.weekday
df.drop('pickup_datetime', axis=1, inplace=True)
# Haversine distance function to calculate ride distance in kilometers
def haversine_distance(lon1, lat1, lon2, lat2):
  lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
  dlon = lon2 - lon1
  dlat = lat2 - lat1
  a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2)**2
  c = 2 * np.arcsin(np.sqrt(a))
  return 6371 * c # 6371 is Earth's radius in km
# Calculate distance and store it
df['distance_km'] = haversine_distance(df['pickup_longitude'], df['pickup_latitude'],
                    df['dropoff_longitude'], df['dropoff_latitude'])
print("--- Task 1: Pre-processing & Feature Engineering Done ---")
# ========================================
# TASK 2: REMOVE OUTLIERS (USING IQR METHOD)
# ========================================
print(f"Total rows before outlier removal: {len(df)}")
# Step A: Basic business rule filtering for impossible records
df = df[df['passenger_count'] > 0]
```

```
df = df[df['passenger_count'] <= 6]
df = df[df['distance_km'] > 0]
df = df[df['distance_km'] <= 100]
# Step B: Statistical IQR Method for 'fare_amount'
Q1 = df['fare_amount'].quantile(0.25)   # 25th percentile (Lower Quartile)
Q3 = df['fare_amount'].quantile(0.75)   # 75th percentile (Upper Quartile)
IQR = Q3 - Q1                           # Interquartile Range
# Calculate mathematical bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
# Ensure the lower bound doesn't go below Uber's minimum base fare ($2.50)
if lower_bound < 2.5:
   lower_bound = 2.5
# Filter the dataframe using these dynamic statistical bounds
df = df[(df['fare_amount'] >= lower_bound) & (df['fare_amount'] <= upper_bound)]
print("--- Task 2: Outliers filtered successfully using the IQR Method! ---")
print(f"Mathematical bounds used -> Lower: ${lower_bound:.2f} | Upper: ${upper_bound:.2f}")
print(f"Total rows remaining: {len(df)}")
# ========================================
# TASK 3: CHECK THE CORRELATION
# ========================================
check_features = ['fare_amount', 'distance_km', 'passenger_count', 'hour', 'weekday', 'year']
print("\n--- Task 3: Correlation Matrix with Target (fare_amount) ---")
print(df[check_features].corr()['fare_amount'].sort_values(ascending=False))
# Plot Heatmap for cleanly engineered columns
plt.figure(figsize=(10, 8))
sns.heatmap(df[check_features].corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Correlation Matrix")
plt.show()
# ========================================
# TASK 4: IMPLEMENT REGRESSION MODELS
```

```
# ========================================
# FIX: Select only meaningful numerical features. Drop raw latitude/longitude columns!
X = df[['distance_km', 'passenger_count', 'hour', 'month', 'year', 'weekday']]
y = df['fare_amount']
# Split Data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("\nTraining data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
# 1. Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
# 2. Random Forest Regression
rf = RandomForestRegressor(n_estimators=50, random_state=42, n_jobs=-1) # Setting estimators to 50 for faster lab running
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
print("--- Task 4: Models Trained Successfully ---")
# ========================================
# TASK 5: EVALUATE AND COMPARE SCORES
# ========================================
r2_lr = r2_score(y_test, y_pred_lr)
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
r2_rf = r2_score(y_test, y_pred_rf)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
results = pd.DataFrame({
  'Model': ['Linear Regression', 'Random Forest Regression'],
  'R2 Score': [r2_lr, r2_rf],
  'RMSE': [rmse_lr, rmse_rf]
})
print("\n================ MODEL COMPARISON ================")
```

```
print(results)
print("================================================")
# ========================================
# DIAGRAMS: ACTUAL VS PREDICTED PLOTS
# ========================================
plt.figure(figsize=(12, 5))

# Linear Regression
plt.subplot(1, 2, 1)
plt.scatter(y_test, y_pred_lr, alpha=0.3, color='blue')
plt.xlabel("Actual Fare")
plt.ylabel("Predicted Fare")
plt.title("Linear Regression - Actual vs Predicted")

# Random Forest
plt.subplot(1, 2, 2)
plt.scatter(y_test, y_pred_rf, alpha=0.3, color='green')
plt.xlabel("Actual Fare")
plt.ylabel("Predicted Fare")
plt.title("Random Forest - Actual vs Predicted")
plt.tight_layout()
plt.show()
# ========================================================
# EXTRA VISUALIZATIONS: SCORES BAR CHART & ERROR HISTOGRAM
# ========================================================
plt.figure(figsize=(14, 5))
# Plot 1: Bar Chart comparing R2 Scores
plt.subplot(1, 2, 1)
model_names = ['Linear Regression', 'Random Forest']
r2_scores = [r2_lr, r2_rf]
sns.barplot(x=model_names, y=r2_scores, palette='Set2')
```

```
plt.ylim(0, 1.0)
plt.title('Model R2 Score Comparison (Higher is Better)')
plt.ylabel('R2 Score')
for i, v in enumerate(r2_scores):
    plt.text(i, v + 0.02, f"{v:.4f}", ha='center', fontweight='bold')
# Plot 2: Histogram of Residuals (Errors)
plt.subplot(1, 2, 2)
lr_residuals = y_test - y_pred_lr
rf_residuals = y_test - y_pred_rf
# Plotting the error distributions
sns.histplot(lr_residuals, color='blue', label='Linear Reg. Errors', kde=True, alpha=0.4, bins=50)
sns.histplot(rf_residuals, color='green', label='Random Forest Errors', kde=True, alpha=0.4, bins=50)
plt.xlim(-15, 15) # Zoom in on the main error zone
plt.title('Distribution of Prediction Errors (Residuals)')
plt.xlabel('Error Value (Actual - Predicted)')
plt.legend()
plt.tight_layout()
plt.show()
```

The **Haversine formula** calculates the shortest straight-line distance between two points on the surface of a sphere using their latitudes and longitudes.

- **Earth is not flat:** Standard geometric distance formulas like Euclidean distance (\(\sqrt{\Delta x^{2}+\Delta y^{2}}\)) assume a completely flat sheet of paper.
- **Coordinates are angles:** Latitude and longitude are angular measurements, not standard grid coordinates. Measuring them with flat-surface math over large spans creates massive distortions.
- **Accuracy:** Uber trips occur on a curved planet. The Haversine formula accounts for the Earth's curvature, providing the actual ground distance in kilometers. This engineered feature gives the machine learning models a direct, high-correlation metric (~0.85) to predict prices accurately.

2. The Mathematical Formula

$$a = \sin^2\left(\frac{\Delta \text{lat}}{2}\right) + \cos(\text{lat}_1) \cdot \cos(\text{lat}_2) \cdot \sin^2\left(\frac{\Delta \text{lon}}{2}\right)$$

$$c = 2 \cdot \arcsin(\sqrt{a})$$

$$d = R \cdot c$$

- **Δlat & Δlon:** The difference between the drop-off and pickup coordinates.
- **a:** The square of half the straight-line chord length between the two points.
- **c:** The angular distance in radians between the points.
- **R:** The mean radius of the Earth (**6371 kilometers**).
- **d:** The final calculated distance on the ground.

Python's trigonometric functions (np.sin, np.cos) do not accept standard degrees (like \(40.71^{\circ }\)). This line uses np.radians to convert all four coordinate inputs from decimal degrees into **radians**. Calculates the absolute change in coordinates (\(\Delta \text{lon}\) and \(\Delta \text{lat}\)) by subtracting the pickup location from the drop-off location. Implements the core Haversine math step. It computes the spherical relationship by factoring in both the raw latitude gaps and the absolute global positions (np.cos(lat1) * np.cos(lat2)).

Computes the inverse sine (arcsin) to yield the actual angular distance between your two points along a great-circle path in radians. Multiplies the angular distance by the Earth's average radius (\(6371\text{ km}\)). This converts the abstract radian value into a real-world unit of measurement (Kilometers) and returns it.

Bar Chart Explanation: *this clearly visualizes our final summary table. The higher bar for Random Forest explicitly shows its superior accuracy and variance capture capability over Linear Regression.* **Histogram Explanation**: *"This shows the distribution of our model errors (Residuals). A good regression model's errors should form a normal distribution bell curve centered tightly around 0. The Random Forest curve is taller and narrower around 0, proving its predictions are consistently closer to the true actual values." Since a perfect prediction equals an error of zero, we want our errors to group as close to zero as possible. Both models peak at 0, but the Random Forest (green) curve is more tightly constrained around the center, proving it has fewer large-magnitude errors compared to Linear Regression (blue).*

*Both Linear Regression and Random Forest Regression are **Supervised Learning** algorithms.**Reason:** They train on a dataset that contains both the input features (\(X\), like distance and time) and a known, labeled target variable (\(y\), the actual fare amount).*

***Simple Definition:** It is a supervised learning method that predicts a target value by drawing a **straight line** through the data points.*

***Simple Formula:** $y=mx+c$ y = Predicted Fare,x= Distance, m = Cost per kilometer (Slope) ,c= Base Fare (Intercept)*

***Fixed Taxi or Delivery Pricing:** Computing basic base-rate logistics charges where the fee is strictly a baseline price plus a fixed cost per mile or kilometer.*

***Retail Commodity Pricing:** Estimating the retail cost of products based directly on manufacturing raw material weight or shipping volume.*

***Real Estate Baseline Evaluation:** Calculating a quick property baseline price using only the raw square footage of a house.*

***Energy Consumption Estimates:** Utility companies predicting monthly electricity usage based strictly on changes in outdoor temperature.*

*It is an ensemble supervised learning method that builds **multiple decision trees** and averages their predictions to get a more accurate final result.* ***Simple Formula:***
*Final Prediction=\frac{\text{Tree}_{1}+\text{Tree}_{2}+\dots +\text{Tree}_{N}}{N}\) .It simply takes the mathematical **average (mean)** of all the individual decision tree outputs*

Random Forest is used when the system must handle **complex, multi-layered data** with sudden changes, peak hours, and unpredictable user behaviors.

Dynamic Ride-Hailing (Uber/Ola Surge Pricing): Adjusting ride fares in real-time by combining distance, live traffic data, passenger demand, driver availability, and time of day.

E-commerce Delivery Time (ETA) Estimation: Predicting exactly when an Amazon package will arrive by analyzing weather conditions, courier speeds, route blockages, and warehouse locations.

Stock Market & Financial Trading: Predicting daily stock price fluctuations or asset values by combining historical trends, trading volume, and global market sentiment.

Credit Score & Loan Risk Assessment: Banks estimating the maximum safe loan amount to give a customer by evaluating income, age, credit history, and current debt levels simultaneously.

- Correlation matrix:: **Definition:** It is a visual grid that shows the **Pearson correlation coefficient** between every pair of features in your dataset.
- **The Scale:** The values range from **-1.0 to +1.0**.
- **Red/Warm colors** indicate a strong positive correlation (as one feature goes up, the other goes up).
- **Blue/Cool colors** indicate a weak or near-zero correlation

---

The Dark Red Diagonal (1.00): Explain to the examiner that the perfect diagonal line of 1.00 values represents each variable correlating perfectly with itself (e.g., fare_amount vs fare_amount). This acts as the baseline check.

The Strongest Relationship (fare_amount vs distance_km = 0.86): Point directly to this box. This is a **strong positive correlation**. It mathematically proves that **distance is the primary driver of the fare price**. As the travel distance increases, the Uber fare increases sharply. This confirms your Haversine formula calculation was highly effective.

The Moderate Relationship (fare_amount vs year = 0.12): This indicates a mild positive trend, showing that over time, base fares have experienced a slight upward shift (likely matching annual inflation or corporate pricing updates).

The Near-Zero Fields (passenger_count, weekday, hour = ~0.01 to -0.02): These fields are colored flat blue. On a strict **linear** scale, features like whether you have 1 or 4 passengers, or whether it is 2 PM vs 3 PM, do not single-handedly scale the baseline price linearly.

IQR stands for Interquartile Range. It is a statistical metric that measures the spread of the middle 50% of your data.

To find it, the data is split into four equal quarters (Quartiles) after sorting it from lowest to highest:

- **Q_{1} (First Quartile / 25th Percentile):** 25% of the fares are below this value.
- **Q_{3} (Third Quartile / 75th Percentile):** 75% of the fares are below this value.
- **Formula:** $\text{IQR}=Q_{3}-Q_{1}$

The bounds act as mathematical fence lines. Any data point that falls outside these fences is classified as an **outlier** and stripped out.

- **Lower Bound:** The minimum limit for normal data.
  $\text{Lower Bound}=Q_{1}-1.5\times \text{IQR}$
  *(In your project, this calculated below zero, so your safety logic clamped it to Uber's minimum base fare of **$2.50**).*
- **Upper Bound:** The maximum limit for normal data.
  $\text{Upper Bound}=Q_{3}+1.5\times \text{IQR}$
  *(In your project, this evaluated dynamically to **$22.25**).*

The Left Line (Whisker): Starts at your Lower Bound ($2.50). It marks the minimum normal fare.

The Left Edge of the Box: Represents Q_{1}.

The Middle Line inside the Box: Represents the Median (50th percentile), which is the exact middle fare of your data.
The Right Edge of the Box: Represents Q_{3}.
The Width of the Box: This physical width *is* your IQR.
The Right Line (Whisker): Stops exactly at your calculated Upper Bound ($22.25). It marks the maximum normal fare.
The Scatter of Dots Outside the Whiskers: These are the outliers (like those long-distance $100 or $500 trips) that sit completely outside your mathematical fences.